

Настройка взаимодействия через Webhook и систему Skills

В данном документе описан процесс создания стабильного канала связи между AI-ассистентом (OpenClaw) и платформой автоматизации (n8n). Это позволяет делегировать выполнение сложных сценариев и работу с внешними API на сторону бэкенда.

1. Архитектура взаимодействия

Система строится на принципе разделения ответственности:

- **OpenClaw Bot:** Выступает в роли интерфейса и оркестратора. Он анализирует запрос пользователя и решает, когда необходимо задействовать внешний инструмент (Skill).
- **Skill:** Программный «мост», который формирует стандартный JSON-запрос и отправляет его на указанный адрес.
- **n8n:** Исполнительная среда. Принимает данные, проверяет права доступа, выполняет бизнес-логику и возвращает результат обратно в чат.

2. Подготовка безопасности

Для защиты вашего вебхука от несанкционированного доступа используется статический ключ (Secret Token).

1. Сгенерируйте сложную строку (рекомендуется использовать 32+ символа).
2. Этот ключ будет передаваться в HTTP-заголовке x-n8n-secret.

Комментарий: В будущем, если вы планируете использовать несколько разных интеграций, лучше хранить этот ключ в переменных окружения (Environment Variables) на стороне Railway или вашего сервера n8n.

3. Настройка сценария в n8n

Шаг 1: Webhook Node

- **Method:** POST
- **Path:** openclaw/skill1
- **Response Mode:** When Last Node Finishes (это важно для корректной передачи ответа обратно в AI).

- **Рекомендация:** Используйте **Production URL** для постоянной работы. Test URL в n8n работает только при открытом окне редактора.

Шаг 2: Проверка авторизации (If Node)

Сразу после вебхука необходимо проверить заголовок. Используйте следующее выражение в условии:

```
{{ $json.headers?["x-n8n-secret"] || $headers["x-n8n-secret"] }}
```

Условие: Is equal to → ВАШ_СЕКРЕТНЫЙ_КЛЮЧ

Шаг 3: Обработка результата

- **Ветка False:** Узел *Respond to Webhook*. Статус 401, тело: {"ok": false, "error": "unauthorized"}. Это предотвратит выполнение сценария при ошибочных запросах.
- **Ветка True:** Узел *Set* (или *Edit Fields*).

Шаг 4: Формирование JSON-ответа

Для того чтобы OpenClaw корректно считал данные, n8n должен вернуть объект. Пример оптимального кода для узла *Set*:

JavaScript

```
{{
  (
    ok: true,
    execution_id: $executionId,
    received: {
      skill: $json.body?.skill,
      args: $json.body?.args
    },
    result: {
      status: "success",
      timestamp: new Date().toISOString(),
      summary: `Действие выполнено для ${$json.body?.args?.username}. Цель:
${$json.body?.args?.goal}`
    }
  )
}}
```

Комментарий: Поле summary крайне полезно. AI использует его, чтобы сформулировать финальный ответ пользователю на естественном языке.

4. Регистрация Skill в OpenClaw

Для активации навыка отправьте ассистенту следующую системную инструкцию:

Инструкция для регистрации:

Register a new skill: "ping".

Endpoint: `https://ваш-адрес.up.railway.app/webhook/openclaw/skill1`

Headers: > - Content-Type: application/json

- x-n8n-secret: ВАШ_СЕКРЕТНЫЙ_КЛЮЧ

Payload structure:

```
{  
  
  "skill": "ping",  
  
  "sessionKey": "<sessionKey>",  
  
  "args": <args>  
}
```

Requirements: Send args as a direct JSON object. Do not wrap in arrays or convert to string.

Промпт для добавления скилов:

Create a new skill named "ping".

Skill purpose:

Send a POST request to an external n8n webhook using JSON.

Skill behavior:

When this skill is invoked, it must send an HTTP POST request with:

- Content-Type: application/json

- Header: x-n8n-secret

Webhook URL:

<https://primary-production-0010c.up.railway.app/webhook/openclaw/skill1>

Header:

x-n8n-secret: 3f9d9c4a1b8e2a77c94c1f8e0d7a5b6a9f3e2d1c4b8a7e9d

Request body format (STRICT, do not modify structure):

```
{  
  "skill": "ping",  
  "sessionKey": "<sessionKey>",  
  "args": <args>  
}
```

Rules:

- sessionKey must be passed dynamically from the conversation context
- args must be a JSON object (not stringified)
- Do NOT stringify args
- Do NOT change field names
- Do NOT add extra fields
- Do NOT wrap payload into arrays
- Always send valid JSON

Implementation:

- Create this skill from scratch
- Register and activate it
- Ensure the skill uses shell / curl (or equivalent HTTP client)
- Provide a short usage example after creation

After creation, confirm that the skill is ready to be used.

5. Рекомендации по эксплуатации

1. **Логирование:** В n8n всегда можно посмотреть историю выполнения (Executions). Если запрос от OpenClaw не дошел, проверьте статус 401 или ошибки в формате JSON.
 2. **Масштабируемость:** Если вы планируете добавить 10–15 новых навыков, не создавайте 15 разных вебхуков. Используйте один, а после него поставьте узел **Switch**, который будет распределять задачи в зависимости от значения поля skill.
 3. **Таймауты:** Помните, что AI-ассистенты обычно ждут ответа от вебхука в течение 30–60 секунд. Если задача в n8n занимает больше времени (например, генерация тяжелого видео), лучше реализовать схему с промежуточным ответом «Принято в работу».
-